

LabReSiD26 – Hands-On 7

Davide Ferrara – 518629

1 Esercizio 1: Osserva il comportamento base

Domanda 1 – Quanti receiver ricevono il messaggio e perché TCP unicast non lo permetterebbe

Eseguendo tre receiver all'interno di Tmux in tre terminali diversi e inviando un messaggio tramite sender, si può notare che tutti i receiver ricevono il messaggio. Con TCP unicast questo non sarebbe possibile perché con esso si stabilisce una connessione 1-a-1 e non 1-a-N come nel caso del multicast: il sender dovrebbe aprire N connessioni separate, una per ogni receiver, e inviare il messaggio N volte. Con multicast è sufficiente una sola `sendto()` — il kernel e la rete si occupano della distribuzione.

Domanda 2 – MAC di destinazione catturato con tcpdump per 239.0.0.1

Output catturato con `sudo tcpdump -i enp8s0 udp port 5000 -e`:

```
1 13:21:03.385919 74:56:3c:c7:5e:5d (oui Unknown) >  
    01:00:5e:00:00:01 (oui Unknown),  
2 ethertype IPv4 (0x0800), length 60:  
3 dave.lan.48598 > 239.0.0.1.complex-main: UDP, length 5
```

Il MAC di destinazione `01:00:5e:00:00:01` corrisponde con quello atteso. Applicando la formula `01:00:5e:XX:XX:XX`: gli ultimi 23 bit di `239.0.0.1` (`EF|00|00|01`) sono `0x000001`, quindi il MAC risultante è `01:00:5e:00:00:01`.

Domanda 3 – Nuovo MAC di destinazione con gruppo 239.1.2.3

Modificando `MCAST_GROUP` in 239.1.2.3 in `sender.c` e `receiver.c`, ricompilando e catturando con `tcpdump`:

```
1 13:31:45.237811 74:56:3c:c7:5e:5d (oui Unknown) >  
    01:00:5e:01:02:03 (oui Unknown),  
2 ethertype IPv4 (0x0800), length 60:  
3 dave.lan.50275 > 239.1.2.3.complex-main: UDP, length 5
```

Il nuovo MAC di destinazione è 01:00:5e:01:02:03. Gli ultimi 23 bit di 239.1.2.3 corrispondono agli ultimi 3 ottetti di `MCAST_GROUP`.

2 Esercizio 2: TTL e loopback

Domanda 1 – Cosa succede con `ttl = 0` e perché

Con `ttl = 0`, `sendto()` ritorna successo e il sender stampa *Inviato (x byte)*..., ma il pacchetto non raggiunge nessuna interfaccia fisica: il kernel lo scarta al livello IP prima che venga consegnato al driver di rete e `tcpdump` sull'interfaccia fisica non cattura nulla.

La differenza tra i due valori è definita qui sotto:

- **TTL = 0** — scope host-local: il pacchetto non esce dalla macchina. Il kernel lo sopprime prima che raggiunga qualsiasi interfaccia di rete. Nessun altro host lo riceve.
- **TTL = 1** — scope link-local: il pacchetto esce sull'interfaccia di rete ma non viene instradato oltre il primo hop. Raggiunge tutti gli host sulla stessa LAN iscritti al gruppo, ma nessun router lo forwarda.

Domanda 2 – Effetto di `IP_MULTICAST_LOOP = 0` sulla stessa macchina e su macchina diversa

Con `loop = 0` il kernel non consegna una copia del pacchetto UDP ai socket locali iscritti al gruppo. Effetto nei due scenari:

- **Stessa macchina:** il receiver non riceve nulla. `IP_MULTICAST_LOOP` sopprime la copia locale prima che venga messa nella coda del socket.
- **Macchina diversa nella stessa LAN:** il receiver riceve normalmente. L'opzione non influenza il pacchetto che esce sull'interfaccia fisica: questo viene inviato sulla rete con `TTL=1` e raggiunge tutti gli host della LAN iscritti al gruppo.

Domanda 3 – Differenza tra `IP_MULTICAST_TTL` e `IP_MULTICAST_LOOP`

Dal manuale:

`IP_MULTICAST_TTL`: Set or read the time-to-live value of outgoing multicast packets for this socket. The default is 1 which means that multicast packets don't leave the local network unless the user program explicitly requests it.

`IP_MULTICAST_LOOP`: Set or read a boolean integer argument that determines whether sent multicast packets should be looped back to the local sockets.

`IP_MULTICAST_TTL` controlla quanti router il pacchetto può attraversare sulla rete. Con `TTL=1` (default) resta nella LAN locale. Con `TTL=0` il kernel sopprime il pacchetto silenziosamente prima di consegnarlo al driver di rete: `sendto()` ritorna successo ma nessun frame esce sull'interfaccia fisica.

`IP_MULTICAST_LOOP` controlla se il kernel consegna una copia del pacchetto ai socket locali prima che esca sulla rete. Non influenza l'uscita del pacchetto sull'interfaccia fisica: con `loop = 0` il pacchetto viene comunque inviato sulla rete, ma nessun socket locale lo riceve.

3 Esercizio 3: Costruisci una chat multicast

Passo A – Un solo socket per inviare e ricevere

chat.c è costruito a partire dalla struttura di receiver.c (bind() + IP_ADD_MEMBERSHIP), viene utilizzato lo stesso socket sia per recvfrom() che per sendto()

Passo B – Il problema del blocco

Mettendo fgets() e recvfrom() in sequenza nel loop:

```
1 while (1) {
2     fgets(buf, sizeof(buf), stdin);      /* blocca finché
3         l'utente scrive */
4     sendto(...);
5     recvfrom(sockfd, buf, ...);          /* blocca finché
6         arriva un pacchetto */
7     printf(...);
8 }
```

Il programma si blocca su fgets() finché l'utente non scrive, e non può ricevere messaggi dalla rete nel frattempo. Simmetricamente, mentre aspetta su recvfrom() non può leggere da tastiera. Le due operazioni bloccanti in sequenza rendono il programma inutilizzabile come chat.

Passo C – select() come soluzione

Con select() su STDIN_FILENO e sockfd ad ogni iterazione il kernel segnala quale dei due fd è pronto e solo quello viene servito:

```
1 while (1) {
2     fd_set readfds;
3     FD_ZERO(&readfds);
4     FD_SET(STDIN_FILENO, &readfds);
5     FD_SET(sockfd, &readfds);
6     select(sockfd + 1, &readfds, NULL, NULL, NULL);
7
8     if (FD_ISSET(STDIN_FILENO, &readfds)) { /* tastiera
9         pronta: invia */ }
10    if (FD_ISSET(sockfd, &readfds)) { /* rete
11        pronta: ricevi */ }
12 }
```

Invio e ricezione funzionano contemporaneamente senza bloccarsi.

Passo D – Il proprio messaggio torna indietro

Per filtrare i propri messaggi è stato adottato il metodo a): il payload viene prefissato con [username] e alla ricezione si controlla se il messaggio inizia con il proprio prefisso:

```
1 /* invio */
2 snprintf(buf, sizeof(buf), "[%s] %s", username, input);
3 sendto(sockfd, buf, strlen(buf), 0, (struct sockaddr *)&
4     local, sizeof(local));
5
6 /* ricezione */
7 char own_prefix[BUFLen];
8 snprintf(own_prefix, sizeof(own_prefix), "[%s]",
9     username);
10 if (strncmp(buf, own_prefix, strlen(own_prefix)) == 0)
11     continue;
```

La differenza rispetto a `IP_MULTICAST_LOOP = 0`, in questo caso si filtra in userspace dopo la ricezione, quindi i propri messaggi arrivano comunque al socket ma vengono scartati.